Universidad Central

Arquitectura de Sistemas I

Plantilla diligenciada

Sección 13.1

Práctica de ingeniería

Adopción responsable de mascotas

Base de datos y persistencia del corte vertical

Proyecto:	Sistema de Apoyo a la Decisión para la Adopción Responsable de Mascotas mediante Inteligencia Artificial
Docente:	Elias Buitrago Bolivar
Integrantes:	Equipo de trabajo (completar nombres)
Fecha:	4 de noviembre de 2026
Estado:	12/12 pruebas de persistencia superadas

Semana 13 · 2026

Contexto del proyecto

Qué es el proyecto. Este trabajo forma parte de la práctica de ingeniería de *Arquitectura de Sistemas I*. El proyecto es un sistema de apoyo a la adopción responsable de mascotas: los refugios registran las fichas de los animales que tienen en adopción y los adoptantes consultan ese catálogo para encontrar una mascota compatible con su hogar. Más adelante el sistema incorporará recomendaciones asistidas por inteligencia artificial; esa parte no se implementa en esta entrega.

Qué se entrega en esta semana. Se prepara la base de datos y la capa de persistencia de dos funciones básicas. Un *corte vertical* es una porción pequeña del sistema que se construye de punta a punta: en esta semana llega hasta la base de datos; la API y la interfaz se completan en las semanas 2 y 3.

Qué significan las siglas.

- **RF** (requisito funcional): una función que el sistema debe cumplir. **RF-05** es la función con la que el refugio crea, edita y da de baja la ficha de una mascota. **RF-34** es la función con la que el adoptante consulta el catálogo de mascotas disponibles y lo filtra por especie, tamaño y edad.
- **RNF** (requisito no funcional): una condición de calidad, no una función. Por ejemplo, rapidez (RNF-05), integridad de los datos (RNF-10), confidencialidad (RNF-04), escalabilidad (RNF-07) y facilidad de prueba (RNF-08).
- **RN** (regla de negocio): una condición propia del dominio. RN-22 exige que la ficha tenga entre 1 y 5 fotos para publicarse; RN-03 exige que el refugio esté verificado; RN-15 retira del catálogo las mascotas adoptadas o dadas de baja.

Por qué se eligieron esos dos RF. RF-05 produce la información y RF-34 la consume. Juntos forman el recorrido mínimo del negocio —publicar una mascota y consultarla— y comparten las tablas *refugio*, *mascota* y *fotografia*, por lo que se pueden probar con una sola base de datos.

Alcance de esta plantilla. Se diligencian los campos de la sección 13.1 «Plantilla de alcance del corte vertical» y se anexa la ficha RF–RNF de la guía de la semana 13. Los identificadores RF-05 y RF-34 se conservan del documento de especificación del proyecto.

1. Plantilla 13.1 — Alcance del corte vertical

Elemento	Descripción del grupo
Funcionalidad priorizada	**RF-05 — Gestión de la ficha de mascota** (prioridad 1): registrar, editar y dar de baja los datos de una mascota. **RF-34 — Consulta del catálogo de mascotas disponibles** (prioridad 2): listar y filtrar las mascotas publicadas. Ambos comparten las tablas *mascota*, *fotografia* y *refugio*.
Actor principal	Refugio (RF-05). Adoptante, refugio y administrador (RF-34). El evaluador no participa en este corte.
Capa de presentación prevista	Formulario web de ficha de mascota para el refugio y catálogo web responsivo para el adoptante. En la semana 1 la verificación se hace por consola y pruebas; la interfaz se integra en la semana 3.
Endpoint(s) requeridos	`POST /api/v1/mascotas` · `PUT /api/v1/mascotas/{id}` · `DELETE /api/v1/mascotas/{id}` · `GET /api/v1/mascotas?especie=&tamano=&edadMin=&edadMax=&pagina=`. Contratos de API-06, API-07, API-08 y API-09 del documento de diseño.
Servicio(s) de aplicación o negocio	*ServicioFichasMascota* (crear, actualizar, dar de baja) y *ServicioCatalogo* (listar y filtrar). En la semana 2 aplican las reglas de negocio; en este corte solo se prepara el contrato del repositorio.
Regla de negocio principal	**RN-22:** una ficha está completa cuando tiene los campos obligatorios y de 1 a 5 fotografías JPG/PNG; solo entonces pasa a *disponible*. **RN-03:** solo un refugio verificado publica fichas. **RN-15:** una ficha adoptada o dada de baja no aparece en el catálogo.
Repositorio o componente de persistencia	**RepositorioMascotas** (`persistencia/repositorio.py`): `crear_ficha`, `obtener_por_id`, `actualizar_ficha`, `dar_de_baja` y `listar_catalogo`. Todo el SQL queda encapsulado aquí.
Tabla(s) o colección(es) involucradas	**refugio** (id, nombre, localidad, estado_verificacion), **mascota** (id, refugio_id, datos de la ficha, estado, fecha_publicacion) y **fotografia** (id, mascota_id, archivo, orden).

Elemento	Descripción del grupo
Validaciones mínimas	Campos obligatorios y dominios cerrados (especie, tamaño, energía, experiencia, estado); edad entre 0 y 300 meses; booleanos 0/1; 1 a 5 fotos JPG/PNG; máximo 5 fotos; orden único por ficha; refugio existente y verificado; FK de refugio y de fotos; transición de estado solo a *disponible* con foto y refugio verificado.
Qué sí se implementaría	Esquema reproducible, carga inicial, repositorio, creación/lectura/actualización/baja de fichas, catálogo con filtros y paginación, restricciones de integridad y 12 pruebas con evidencia. Lectura desde una conexión nueva y rechazo de id inexistente.
Qué queda fuera del alcance	Autenticación completa y matriz de permisos; interfaz web; almacenamiento binario de fotos; motor de recomendaciones (RF-06 a RF-09); solicitudes y decisiones de adopción (RF-30, RF-31); seguimiento posadopción; notificaciones; indicadores; chatbot y reconocimiento de imágenes. La migración definitiva a PostgreSQL se hace en la semana 2.
Riesgo técnico principal	Consistencia entre la ficha, sus fotografías y el estado de publicación. Se mitiga con transacción (*BEGIN IMMEDIATE* + *ROLLBACK*), validación en el repositorio y triggers en la base. El almacenamiento de fotos se limita a metadatos/ruta en este corte.

2. Priorización de los dos RF

Recordatorio: RF-05 es la función de publicar y mantener una mascota; RF-34 es la función de consultar el catálogo de mascotas disponibles.

Orden	RF y resultado observable	Valor para el actor	Relevancia arquitectónica	Esfuerzo, riesgo o dependencia que explica el orden
1	**RF-05.** El refugio crea, actualiza o da de baja una ficha de mascota y obtiene su identificador y estado (*incompleta* o *disponible*).	Es la operación que produce el contenido del sistema: sin fichas no hay catálogo ni recomendaciones. Alto valor para el refugio.	Define el modelo de escritura, la integridad referencial, la relación refugio–mascota–foto y la regla de publicación. Es la base de los demás módulos.	No tiene dependencias funcionales. Esfuerzo medio por la validación de completitud y las fotografías. Riesgo principal: dejar fichas a medias; se controla con transacción y triggers.
2	**RF-34.** El adoptante, el refugio o el administrador consultan una lista paginada de mascotas disponibles, con filtros por especie, tamaño y edad, de la más antigua a la más reciente.	Es la vía de selección tradicional y el punto de entrada del adoptante. Alto valor, pero depende de que existan fichas publicadas.	Define el modelo de lectura, el orden estable, la paginación y los índices. Su desempeño se vuelve crítico al crecer el catálogo.	Depende de RF-05: no se puede consultar lo que aún no se ha creado. Menor esfuerzo que RF-05. Riesgo: degradación con 2.000 fichas; se mitiga con índices y prueba de carga en semana 3.

Fuera del alcance de este corte: el motor de recomendaciones, las solicitudes de adopción, el seguimiento posadopción, las notificaciones y los indicadores. Se retomarán después de estabilizar la persistencia de las fichas y el catálogo.

3. Matriz RF–RNF (ISO/IEC 25010:2023)

Cada RNF se asocia de forma explícita a por lo menos un RF. En la columna *RNF / RF asociados*, RF-05 corresponde a la ficha de la mascota y RF-34 al catálogo. Los umbrales los define el equipo según el contexto del proyecto; la norma orienta la característica y la subcaracterística, no fija los valores. Se indica qué se verificó en la semana 1 y qué queda para las semanas 2 y 3.

RNF / RF asociados	Característica y subcaracterística ISO/IEC 25010:2023	Condición de prueba	Medida y umbral	Evidencia y semana de verificación
RNF-05 RF-05, RF-34	**Eficiencia de desempeño /** Comportamiento temporal	Entorno local documentado; 500 fichas activas y 50 usuarios concurrentes; 20 consultas consecutivas al catálogo, con y sin filtros.	p95 ≤ 1 s por consulta; máximo ≤ 1,5 s; 0 errores de consulta.	*Parcial.* Semana 1: línea base con 8 fichas (0,53 ms promedio; 0,66 ms máximo). Semana 2: medición sobre la API. Semana 3: carga con 500 fichas. Evidencia: `salida_pruebas.txt`, prueba 12.
RNF-10 RF-05, RF-34	**Seguridad /** Integridad	Matriz de entradas inválidas: tipo, rango, valor no permitido, refugio no verificado, ficha sin foto, más de 5 fotos y FK rota. Verificar antes y después de cada intento.	100 % de las entradas inválidas se rechaza con ERR-004, ERR-006 o ERR-009 y 0 filas quedan persistidas; ninguna ficha incompleta aparece en el catálogo.	*Parcial.* Semana 1: 6 casos ejecutados (pruebas 2, 3, 5, 6, 9 y 10), todos rechazados. Semana 2: matriz completa sobre la API. Evidencia: `salida_pruebas.txt`.
RNF-08 RF-05, RF-34	**Mantenibilidad /** Capacidad de prueba	Reemplazar el repositorio real por un doble en los servicios de la semana 2 y ejecutar una prueba de resultado válido y otra de rechazo por cada operación, sin base real.	2 pruebas por operación (válida y de rechazo), repetibles; cobertura ≥ 70 % en los módulos de reglas de negocio y servicios.	*Parcial.* Semana 1: contrato del repositorio definido y 12 pruebas directas. Semana 2: dobles de repositorio y cobertura. Evidencia: `repositorio.py` y `salida_pruebas.txt`.
RNF-07 RF-34	**Flexibilidad /** Escalabilidad	Catálogo con 2.000 fichas activas y 5.000 adoptantes; agregar un refugio o una localidad sin modificar código; consultas con filtros y paginación.	p95 ≤ 1 s; 0 cambios de código al incorporar un refugio o una localidad nuevos.	*Pendiente.* Semana 3: prueba de carga y revisión de índices. No se declara cumplido en este corte.
RNF-04 RF-05, RF-34	**Seguridad /** Confidencialidad	Revisar las claves de la respuesta del catálogo y de la ficha pública: no debe aparecer correo, teléfono, nombre completo ni dirección del adoptante.	0 campos personales en el 100 % de las respuestas del catálogo; solo nombre y localidad del refugio.	*Parcial.* Semana 1: prueba 11, 0 campos personales en catálogo y ficha. Semana 2: matriz de permisos por rol y endpoints. Evidencia: `salida_pruebas.txt`.

4. Ficha RF–RNF del equipo

4.1. Ficha de RF-05 — Gestión de la ficha de mascota

Elemento	Respuesta del equipo
Identificador y enunciado del RF	**RF-05.** El sistema debe permitir al refugio crear, editar y dar de baja fichas de mascotas, con especie, raza aproximada, edad, tamaño, nivel de energía, historial de salud y comportamiento, experiencia requerida, aptitud para convivir con menores y otras mascotas, y de 1 a 5 fotografías.
Actor y resultado esperado	Refugio. Obtiene el identificador de la ficha y su estado: *incompleta* si falta algún campo o fotografía, o *disponible* si cumple RN-22 y el refugio está verificado.
Prioridad y justificación: valor, relevancia arquitectónica y dependencia	**Prioridad 1.** Alto valor: produce el contenido que alimenta el catálogo y el motor de recomendaciones. Relevancia arquitectónica alta: fija el modelo de escritura, la integridad referencial y la relación refugio–mascota–foto. No tiene dependencias funcionales; su esfuerzo es medio por la validación de completitud y las fotografías.
RNF asociados e identificadores	RNF-04 (Seguridad / confidencialidad), RNF-05 (Eficiencia de desempeño / comportamiento temporal), RNF-08 (Mantenibilidad / capacidad de prueba) y RNF-10 (Seguridad / integridad).
Característica y subcaracterística ISO/IEC 25010	Eficiencia de desempeño → comportamiento temporal; Seguridad → integridad y confidencialidad; Mantenibilidad → capacidad de prueba.
Condición, medida y umbral de cada RNF	**RNF-05:** 500 fichas y 50 usuarios; creación y lectura p95 $\leq$ 1 s. **RNF-10:** 100 % de entradas inválidas rechazadas y 0 filas persistidas. **RNF-08:** 2 pruebas por operación (válida y rechazo), cobertura $\geq$ 70 %. **RNF-04:** 0 datos personales en la ficha pública.
Prueba, evidencia y semana de verificación	Semana 1: pruebas 1, 2, 3, 5, 6, 8, 9, 10 y 11 de `salida_pruebas.txt`. Semana 2: API, autenticación por rol y pruebas de servicio con doble del repositorio. Semana 3: integración con la interfaz.
Datos y operaciones de persistencia	Tablas **refugio**, **mascota** y **fotografia**. Operaciones: `crear_ficha`, `obtener_por_id`, `actualizar_ficha` y `dar_de_baja`.
Resultado de las pruebas de semana 1 y pendientes	**12/12 pruebas superadas.** Crear y leer desde conexión nueva, rechazo de refugio no verificado, datos inválidos, FK rota, ficha incompleta, más de 5 fotos y triggers de publicación. Pendiente: autenticación, fotos binarias, actualización campo a campo, pruebas de servicio y migración a PostgreSQL.

4.2. Ficha de RF-34 — Consulta del catálogo de mascotas disponibles

Elemento	Respuesta del equipo
Identificador y enunciado del RF	**RF-34.** Cualquier usuario autenticado, salvo el evaluador, puede listar las mascotas en estado disponible, ordenadas por fecha de publicación (la más antigua primero) y filtrar por especie, tamaño y rango de edad. El catálogo no muestra puntaje, explicación ni riesgo.
Actor y resultado esperado	Adoptante, refugio y administrador. Obtienen una lista paginada de 20 fichas resumidas por página, con id, nombre, especie, tamaño, edad, fecha de publicación, foto principal y refugio (nombre y localidad).
Prioridad y justificación: valor, relevancia arquitectónica y dependencia	**Prioridad 2.** Alto valor para el adoptante: es la vía de selección tradicional. Relevancia arquitectónica media-alta: define el modelo de lectura, el orden estable, la paginación y los índices. Depende de RF-05, porque no se puede consultar una ficha que aún no existe; por eso se implementa después.
RNF asociados e identificadores	RNF-04 (Seguridad / confidencialidad), RNF-05 (Eficiencia de desempeño / comportamiento temporal), RNF-07 (Flexibilidad / escalabilidad), RNF-08 (Mantenibilidad / capacidad de prueba) y RNF-10 (Seguridad / integridad).
Característica y subcaracterística ISO/IEC 25010	Eficiencia de desempeño → comportamiento temporal; Flexibilidad → escalabilidad; Seguridad → confidencialidad e integridad; Mantenibilidad → capacidad de prueba.
Condición, medida y umbral de cada RNF	**RNF-05:** 500 fichas, 50 usuarios, p95 ≤ 1 s. **RNF-07:** 2.000 fichas y 5.000 adoptantes, p95 ≤ 1 s, sin cambios de código al agregar refugio o localidad. **RNF-04:** 0 datos personales en la respuesta. **RNF-10:** 100 % de filtros inválidos rechazados y 0 resultados incorrectos.
Prueba, evidencia y semana de verificación	Semana 1: pruebas 4, 7, 11 y 12 de `salida_pruebas.txt` (id inexistente, orden, filtros, paginación, datos personales y línea base de tiempos). Semana 2: endpoint GET /mascotas. Semana 3: prueba de carga con 2.000 fichas e interfaz.
Datos y operaciones de persistencia	Tablas **mascota**, **fotografia** y **refugio**. Operación: `listar_catalogo(especie, tamano, edad_min, edad_max, pagina, por_pagina)`. Índices: `idx_mascota_catalogo` y `idx_mascota_filtros`.
Resultado de las pruebas de semana 1 y pendientes	**Catálogo correcto:** total 4, orden [Luna, Rocky, Milo, Nala], perros 3, pequeños 1, página 2 con 1 elemento. Línea base: 0,53 ms promedio y 0,66 ms máximo en 20 consultas. Pendiente: prueba de carga con 2.000 fichas, endpoint, interfaz y verificación de índices en PostgreSQL.

5. Persistencia: modelo, operaciones y resultados

5.1. Modelo mínimo de datos

Entidad	Clave	Campos principales	Relación	RF
refugio	id (PK)	nombre, localidad, estado_verificacion (pendiente / verificado / rechazado)	1 : N con mascota	RF-05, RF-34
mascota	id (PK)	refugio_id (FK), nombre, raza, especie, edad_meses, tamano, nivel_energia, experiencia_requerida, apta_con_menores, apta_con_mascotas, historial_salud, historial_comportamiento, estado (incompleta / disponible / adoptada / baja), fecha_publicacion	N : 1 con refugio; 1 : N con fotografia	RF-05, RF-34
fotografia	id (PK)	mascota_id (FK), archivo, orden (1 a 5)	N : 1 con mascota; UNIQUE (mascota_id, orden)	RF-05, RF-34

Cambios frente al modelo previo: no se agregan entidades nuevas. Se especializa *mascota* con *estado* y *fecha_publicacion*, y *fotografia* con *orden* único. Se conservan las relaciones del modelo del segundo corte. En la base local las fotos se guardan como ruta; el binario se incorporará en la semana 2.

5.2. Contrato del repositorio

Operación	Entrada	Resultado	Uso
`crear_ficha`	refugio_id, ficha de RN-22, 0–5 fotos	id real; estado incompleta o disponible	RF-05: persistir y devolver el identificador.
`obtener_por_id`	mascota_id	ficha con refugio y fotos, o None	RF-05: verificar el registro; representar ausencia.
`actualizar_ficha`	mascota_id, refugio_id, cambios, fotos opcionales	ficha actualizada y estado recalculado	RF-05: editar y volver a publicar.
`dar_de_baja`	mascota_id, refugio_id	baja lógica (estado = baja)	RF-05: retirar la ficha del catálogo.
`listar_catalogo`	especie, tamano, edad_min, edad_max, pagina, por_pagina	total, pagina, porPagina, mascotas	RF-34: consulta paginada y filtrada.

5.3. Reglas de integridad y separación de capas

- **FK** refugio → mascota y mascota → fotografia (cascada en fotos).
- **CHECK** de dominios: especie, tamaño, energía, experiencia, estado y edad 0–300.
- **UNIQUE** (mascota_id, orden) y máximo 5 fotos con trigger.
- **Triggers**: disponible exige foto y refugio verificado (RN-03/RN-22).
- **Transacción**: *BEGIN IMMEDIATE*; ante error, *ROLLBACK*.
- **Repositorio**: único componente con SQL; la API y los servicios no consultan directamente.
- **Catálogo**: solo fichas disponibles de refugios verificados; sin datos personales del adoptante (RN-24).
- **Pendiente semana 2**: decidir quién confirma la transacción en la API y cómo se liberan las conexiones.

5.4. Datos de prueba y estado inicial

Mascota	Refugio	Especie / tamaño	Estado	¿Catálogo?	Motivo
1 — Luna	1 — Patitas (verificado)	perro / grande	disponible	**Sí**	Ficha completa.
2 — Milo	1 — Patitas (verificado)	gato / pequeño	disponible	**Sí**	Ficha completa.
3 — Rocky	1 — Patitas (verificado)	perro / mediano	disponible	**Sí**	Ficha completa.
4 — Nube	2 — Colitas (pendiente)	gato / pequeño	disponible	*No*	Refugio no verificado.
5 — Toby	1 — Patitas (verificado)	perro / pequeño	incompleta	*No*	Sin fotografía.
6 — Canela	1 — Patitas (verificado)	perro / mediano	adoptada	*No*	Ya adoptada.

Orden esperado del catálogo con los datos iniciales: **Luna (2026-10-01), Rocky (2026-10-03), Milo (2026-10-05)**. La prueba 1 agrega a Nala (id 7, 2026-10-10), por lo que la prueba 7 obtiene **[Luna, Rocky, Milo, Nala]**. La fecha de referencia del corte es 2026-11-04.

5.5. Resultados de las pruebas de persistencia

#	Prueba ejecutada	Resultado
1	Crear ficha en refugio verificado y leerla desde una conexión nueva.	**PASA — id 7, disponible, 2 fotos.**
2	Rechazar ficha de refugio no verificado.	**PASA — ERR-009; 0 filas nuevas.**
3	Rechazar datos inválidos (especie no permitida).	**PASA — ERR-004; 0 filas nuevas.**
4	Consultar un identificador inexistente.	**PASA — devuelve None.**
5	Violar la FK de refugio.	**PASA — FOREIGN KEY constraint failed.**
6	Ficha sin fotos queda incompleta y fuera del catálogo.	**PASA — id 8, estado incompleta.**
7	Catálogo: solo disponibles/verificados; orden, filtros y paginación.	**PASA — total 4, orden [1,3,2,7].**
8	Actualizar ficha incompleta, publicarla y darla de baja.	**PASA — incompleta → disponible → baja.**
9	Rechazar más de 5 fotografías.	**PASA — repositorio y trigger.**
10	Triggers de publicación: refugio pendiente y ficha sin foto.	**PASA — ambos rechazos.**
11	El catálogo y la ficha no exponen datos personales.	**PASA — 0 campos personales.**
12	20 consultas consecutivas al catálogo.	**PASA — 0,53 ms promedio; 0,66 ms máximo.**

Resumen: 12/12 pruebas superadas. Python 3.12.10 y SQLite 3.49.1. Evidencia completa: `evidencias/salida_pruebas.txt` y `evidencias/prueba.db`. La medición de tiempos es una línea base con 8 fichas; no sustituye la prueba de carga de RNF-05.

6. Reproducción, evidencia y cierre

6.1. Requisitos y comandos exactos

Requisitos

- Python 3.12.10 (compatible con 3.10 o superior).
- SQLite 3.49.1 incluido en Python; sin servidor ni dependencias externas.
- Sin credenciales: la base local no usa usuario ni contraseña.

La base local se usa para que cualquier integrante reproduzca el corte sin instalar PostgreSQL. El esquema conserva FK, CHECK, UNIQUE, índices y triggers; la migración a PostgreSQL (ADR-03/ADR-10) se hace en la semana 2.

Comandos (PowerShell)

```
cd "C:\...\corte_vertical_sem13\persistencia"
$env:PYTHONUTF8 = "1"
$env:PYTHONIOENCODING = "utf-8"
python prueba_persistencia.py
python demo_consulta.py
```

La prueba recrea la base, ejecuta los 12 casos y escribe la evidencia.

6.2. Salida esperada y ubicación de evidencias

Evidencia	Ubicación / contenido
Resultado de pruebas	`evidencias/salida_pruebas.txt` — 12/12 PASA y tiempos.
Base reproducible	`evidencias/prueba.db` — recreada por `schema.sql` + `seed.sql`.
Consulta manual	`evidencias/salida_consulta.txt` — catálogo por especie y ficha 1.
Código de persistencia	`persistencia/repositorio.py` y `persistencia/schema.sql`.
Instrucciones completas	`README.md` del corte vertical.
Plantilla editable	`plantilla/plantilla_13_1.tex` — se compila en Overleaf con pdfLaTeX.

6.3. Checklist de revisión

Criterio de revisión	Cumple	Evidencia
Hay dos RF identificados, priorizados y justificados.	**Sí**	Sección 2.
Cada RF se vincula con sus RNF e ISO/IEC 25010.	**Sí**	Sección 3 y fichas 4.1/4.2.
Los RNF incluyen condición, medida, umbral y verificación.	**Sí**	Sección 3; lo pendiente está marcado.
Los datos y operaciones cubren ambos RF.	**Sí**	Secciones 4 y 5.
La base y los datos de prueba pueden recrearse.	**Sí**	`schema.sql`, `seed.sql` y comando de 6.1.
Se demuestra lectura, escritura y persistencia real.	**Sí**	Pruebas 1, 6, 7 y 8; conexión nueva.
El README indica requisitos y pasos exactos de prueba.	**Sí**	`README.md` y sección 6.
Se distingue lo probado de lo pendiente para semanas 2 y 3.	**Sí**	Secciones 3, 4 y 5.5; pendientes explícitos.

6.4. Cierre

El corte construye primero **RF-05** porque produce las fichas que alimentan el catálogo y fija las reglas de integridad; luego **RF-34**, que lee esas fichas y permite verificar el registro desde el punto de vista del adoptante. La calidad esperada se concentra en integridad de datos, comportamiento temporal y capacidad de prueba. La evidencia que permite verificarla es la base recreable, el repositorio con SQL encapsulado, las 12 pruebas ejecutadas y la salida con tiempos. La semana 2 continúa sobre estas mismas decisiones: API, reglas de negocio, autenticación por rol y pruebas con dobles del repositorio.